



ESCUELA POLITÉCNICA NACIONAL

ESCUELA DE FORMACIÓN DE TECNÓLOGOS



ESCUELA POLITÉCNICA NACIONAL
ESFOT – TECNOLOGÍA SUPERIOR EN
DESARROLLO DE SOFTWARE

PROGRAMACIÓN
ORIENTADA A OBJETOS
JAVA



DOCENTE:
Ing. Yadira
Franco Rocha



TAREA INTEGRADORA SISTEMA DE GESTIÓN DE PERSONAL PARA CLÍNICA PRIVADA

FASE 1 – INVESTIGACIÓN TEÓRICA (30%)

1. ¿QUÉ SON EXCEPCIONES EN JAVA?

- Concepto
- Por qué existen
- Cuándo aplicarlas
- Diferencia entre errores lógicos y excepciones

EJEMPLO

EXCEPCIÓN

```
String edad = "abc";  
int numero =  
Integer.parseInt(edad);
```

Resultado:

NumberFormatException

ERROD LÓGICO

```
int resultado = 5 / 2;  
System.out.println(resultado);
```

Resultado:

2

El programa no se detiene, pero el resultado no es el esperado.

3. CONVERSIONES EN JAVA

String → int

```
Integer.parseInt()
```

Ejemplo:

```
String edad = "25";  
int e = Integer.parseInt(edad);
```

String → double

```
Double.parseDouble()
```

Ejemplo:

```
String valor = "15.75";  
double precio =  
Double.parseDouble(valor);
```

Conversión explícita (casting)

```
(int) (double)
```

Ejemplo:

```
double promedio = 9.8;  
int nota = (int) promedio; // 9
```

TIPOS DE CONVERSIONES

- Implícita (automática)
- Explícita (casting)
- Riesgos de pérdida de información

Ejemplo:

```
double numero = 15.99;  
int entero = (int) numero;
```

Resultado: 15

Se pierde: 0.99

2. TIPOS DE EXCEPCIONES

- NumberFormatException
- InputMismatchException
- ArithmeticException
- NullPointerException
- IndexOutOfBoundsException
- IllegalArgumentException
- Exception

PARA CADA EXCEPCIÓN:

- ✓ Definición
- ✓ Ejemplo
- ✓ Cuándo ocurre
- ✓ Cómo evitarla
- ✓ Cuándo aplicarla



PALABRAS CLAVE PRINCIPALES

try	Encierra el código que puede fallar.	Intentar hacer algo.
catch	Captura y controla el error.	Si falla, responder con mensaje.
finally	Se ejecuta siempre.	Cerrar o finalizar proceso.
throw	Lanza un error manualmente.	Yo provooco el error.
throws	Avisa que un método puede lanzar error.	Este método puede fallar.

ENTREGABLE TEÓRICO



- Documento PDF con:
- ✓ Portada
 - ✓ Índice
 - ✓ Investigación completa
 - ✓ Ejemplos explicados
 - ✓ Conclusiones personales

FASE 2 – DESARROLLO PRÁCTICO (70%)

CASO REAL Sistema de Gestión de Personal para Clínica Privada

Una clínica privada requiere automatizar el registro y control de su personal.

El sistema gestionará:



REQUERIMIENTOS FUNCIONALES

Código	Funcionalidad
RF01	Registrar médicos
RF02	Registrar administrativos
RF03	Mostrar registros
RF04	Buscar por cédula
RF05	Reemplazar información
RF06	Eliminar registros
RF07	Calcular pagos
RF08	Mostrar estadísticas
RF09	Salir

MODELO POO



CÁLCULOS OBLIGATORIOS



MÉDICO

pago = pacientes * valor consulta

Ejemplo:

40 * 25 = 1000



ADMINISTRATIVO

pago = horas * valor hora

Ejemplo:

160 * 4.50 = 720

CRUD OBLIGATORIO

- CREATE** Registrar empleados.
- READ** Mostrar empleados.
- UPDATE** Reemplazar datos.
- DELETE** Eliminar registros.

MENÚ OBLIGATORIO

1. Registrar médico
2. Registrar administrativo
3. Mostrar empleados
4. Buscar por cédula
5. Reemplazar información
6. Eliminar registro
7. Calcular pagos
8. Mostrar estadísticas
9. Salir

VALIDACIONES OBLIGATORIAS

- ✗ Menú inválido (letras, números fuera de rango)
- ✗ Edad (no permitir: abc, -1, 0, 150)
- ✗ Cédula duplicada
- ✗ Campos vacíos (nombre, correo, especialidad, departamento)
- ✗ Correo debe contener @ y .
- ✗ Teléfono solo números
- ✗ Pacientes atendidos > 0
- ✗ Valor consulta > 0
- ✗ Horas trabajadas > 0
- ✗ Valor por hora > 0

¿QUÉ HACER SI INGRESAN ERRORES?

Error	Acción
Letras en menú	Mostrar error
Texto en edad	Capturar excepción
Cédula repetida	Bloquear registro
Búsqueda inexistente	Informar
Valores negativos	Volver a pedir



EXCEPCIONES A APLICAR

- Usar try-catch para:
- Menú
 - Conversiones
 - Datos numéricos
 - Búsquedas
 - Listas



CONVERSIONES OBLIGATORIAS

- Aplicar:
- Integer.parseInt()
 - Double.parseDouble()



POLIMORFISMO OBLIGATORIO

- Usar:
- ```
ArrayList<Empleado>
```

#### ESTRUCTURA DE PAQUETES



#### RESPONSABILIDADES

- modelo** Clases del negocio.
- servicio** CRUD.
- util** Validaciones.
- app** Menú principal.

#### UML OBLIGATORIO (Ejemplo)



#### ESTADÍSTICAS A MOSTRAR

- Total médicos
- Total administrativos
- Total empleados
- Pago total médicos
- Pago total administrativos
- Empleado con mayor ingreso



#### GITHUB OBLIGATORIO

Subir proyecto con mínimo 5 commits.



#### README OBLIGATORIO

- Explicar:
- ✓ Herencia
  - ✓ Encapsulamiento
  - ✓ Polimorfismo
  - ✓ Excepciones
  - ✓ Conversiones
  - ✓ Validaciones



#### ENTREGABLES FINALES

- ✓ PDF teórico
- ✓ Proyecto Java
- ✓ UML
- ✓ Capturas del sistema
- ✓ GitHub (min. 5 commits)
- ✓ README

# INVESTIGACIÓN TEÓRICA

## Programación Orientada a Objetos – Java

### 1. ¿Qué son las Excepciones en Java?

#### Concepto

Una excepción en Java es un evento inesperado que interrumpe el flujo normal de ejecución de un programa. Cuando ocurre un error en tiempo de ejecución (por ejemplo, dividir entre cero o convertir texto no numérico a número), la JVM crea un objeto de excepción con información sobre el problema y lo "lanza" para que el programa pueda manejarlo de forma controlada.

#### ¿Por qué existen?

Las excepciones existen para separar el código de lógica normal del código de manejo de errores. Sin excepciones, cada operación requeriría comprobaciones manuales de error, haciendo el código difícil de leer y mantener. Las excepciones permiten al programador anticiparse a fallos predecibles y responder de manera ordenada sin que el programa colapse.

#### ¿Cuándo aplicarlas?

- Al leer datos del usuario que pueden contener letras donde se esperan números.
- Al convertir tipos de datos ( $\text{String} \rightarrow \text{int}$ ,  $\text{String} \rightarrow \text{double}$ ).
- Al acceder a índices de listas que podrían estar fuera de rango.
- Al trabajar con objetos que podrían ser nulos (null).
- Al realizar operaciones matemáticas que podrían dar error (división entre cero).

#### Diferencia entre Error Lógico y Excepción

| Característica    | Error Lógico                                       | Excepción                                                                 |
|-------------------|----------------------------------------------------|---------------------------------------------------------------------------|
| ¿Se detiene?      | No. El programa continúa.                          | Sí. El programa se detiene si no se captura.                              |
| ¿El resultado?    | Incorrecto pero sin error visible.                 | Lanza un mensaje de error (stack trace).                                  |
| ¿Se detecta?      | Solo revisando la lógica manualmente.              | Java la detecta automáticamente.                                          |
| Ejemplo           | <code>int r = 5/2;</code> → da 2 (debería ser 2.5) | <code>Integer.parseInt("abc")</code> → <code>NumberFormatException</code> |
| ¿Cómo se corrige? | Revisando el algoritmo.                            | Con try-catch.                                                            |

## Tipos de Excepciones en Java

Para cada excepción se indica: definición, ejemplo de código, cuándo ocurre, cómo evitarla y cuándo aplicarla.

| NumberFormatException |                                                                                                               |
|-----------------------|---------------------------------------------------------------------------------------------------------------|
| Definición            | Se produce cuando se intenta convertir un texto (String) a un número y el contenido no es numérico válido.    |
| Ejemplo               | <code>String valor = "hola"; int numero = Integer.parseInt(valor); // Resultado: NumberFormatException</code> |
| ¿Cuándo ocurre?       | Cuando el usuario ingresa letras o símbolos donde el sistema espera un número entero o decimal.               |
| ¿Cómo evitarla?       | Con bloque try-catch capturando NumberFormatException. Validar antes de convertir si es posible.              |
| ¿Cuándo aplicarla?    | En el menú, al leer edad, pagos, horas trabajadas, número de pacientes.                                       |

| InputMismatchException |                                                                                                                                            |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Definición             | Ocurre cuando el Scanner espera un tipo de dato específico (por ejemplo int) pero el usuario ingresa otro tipo (letras).                   |
| Ejemplo                | <code>Scanner sc = new Scanner(System.in); int numero = sc.nextInt(); // usuario escribe "abc" // Resultado: InputMismatchException</code> |
| ¿Cuándo ocurre?        | Cuando se usa nextInt(), nextDouble() del Scanner y el usuario escribe texto en lugar de número.                                           |
| ¿Cómo evitarla?        | Usar nextLine() para leer como String y luego convertir manualmente con parseInt/parseDouble.                                              |
| ¿Cuándo aplicarla?     | En cualquier lectura de datos numéricos del usuario mediante Scanner.                                                                      |

| ArithmeticException |                                                                                                             |
|---------------------|-------------------------------------------------------------------------------------------------------------|
| Definición          | Se lanza cuando ocurre un error matemático, el más común siendo la división entre cero con números enteros. |
| Ejemplo             | <code>int a = 10; int b = 0; int resultado = a / b; // Resultado: ArithmeticException: / by zero</code>     |
| ¿Cuándo ocurre?     | Cuando el divisor en una operación entera es cero. Con doubles no ocurre (da Infinity).                     |
| ¿Cómo evitarla?     | Verificar que el divisor sea distinto de cero antes de dividir, o capturar la excepción.                    |
| ¿Cuándo aplicarla?  | En cálculos de promedios, porcentajes, o cualquier división donde el denominador puede ser cero.            |

| NullPointerException |                                                                                   |
|----------------------|-----------------------------------------------------------------------------------|
| Definición           | Se produce cuando se intenta usar un objeto cuya referencia es null, es decir, no |

|                           |                                                                                                       |
|---------------------------|-------------------------------------------------------------------------------------------------------|
|                           | apunta a ningún objeto en memoria.                                                                    |
| <b>Ejemplo</b>            | <code>String nombre = null; int longitud = nombre.length(); // Resultado: NullPointerException</code> |
| <b>¿Cuándo ocurre?</b>    | Al llamar métodos o acceder a atributos de un objeto que no fue inicializado (es null).               |
| <b>¿Cómo evitarla?</b>    | Verificar que el objeto no sea null antes de usarlo: <code>if (objeto != null) { ... }</code>         |
| <b>¿Cuándo aplicarla?</b> | En búsquedas (verificar si el resultado es null antes de mostrar), listas, objetos opcionales.        |

| IndexOutOfBoundsException |                                                                                                                                                           |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Definición</b>         | Se lanza cuando se intenta acceder a un índice que no existe en un array o ArrayList (fuera del rango válido).                                            |
| <b>Ejemplo</b>            | <code>ArrayList&lt;String&gt; lista = new ArrayList&lt;&gt;(); lista.add("Juan"); String e = lista.get(5); // Resultado: IndexOutOfBoundsException</code> |
| <b>¿Cuándo ocurre?</b>    | Cuando el índice usado es negativo o mayor o igual al tamaño de la colección.                                                                             |
| <b>¿Cómo evitarla?</b>    | Verificar el tamaño con <code>.size()</code> o <code>.length</code> antes de acceder. Usar bucles for-each cuando sea posible.                            |
| <b>¿Cuándo aplicarla?</b> | En recorridos de listas, acceso a posiciones específicas de arrays.                                                                                       |

| IllegalArgumentException  |                                                                                                                                                                    |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Definición</b>         | Se lanza cuando un método recibe un argumento con un valor que no es aceptable o está fuera del rango permitido.                                                   |
| <b>Ejemplo</b>            | <code>public void setEdad(int edad) { if (edad &lt;= 0    edad &gt;= 150) throw new IllegalArgumentException("Edad inválida: " + edad); this.edad = edad; }</code> |
| <b>¿Cuándo ocurre?</b>    | Cuando el programador la lanza manualmente con throw al detectar un valor de entrada no permitido.                                                                 |
| <b>¿Cómo evitarla?</b>    | Validar los datos antes de pasarlos al método, o capturar la excepción con try-catch.                                                                              |
| <b>¿Cuándo aplicarla?</b> | En validaciones de negocio: edades, valores negativos, rangos de datos.                                                                                            |

| Exception (clase base)    |                                                                                                                                                         |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Definición</b>         | Es la clase padre de todas las excepciones comprobadas en Java. Se usa como captura genérica cuando no se sabe exactamente qué excepción puede ocurrir. |
| <b>Ejemplo</b>            | <code>try { // código que puede fallar } catch (Exception e) { System.out.println("Error: " + e.getMessage()); }</code>                                 |
| <b>¿Cuándo ocurre?</b>    | Cuando cualquier excepción comprobada no es capturada por un catch más específico.                                                                      |
| <b>¿Cómo evitarla?</b>    | Usarla como último recurso en una cadena de catch, después de los tipos específicos.                                                                    |
| <b>¿Cuándo aplicarla?</b> | Como captura genérica de respaldo en operaciones críticas o desconocidas.                                                                               |

## Conversiones en Java

### Conversión String → int con Integer.parseInt()

Permite convertir texto que representa un número entero en un valor de tipo int. Es fundamental para leer datos del usuario (que siempre llegan como String) y convertirlos a números para operar con ellos.

Ejemplo:

```
String edad = "25";
int e = Integer.parseInt(edad);
// e = 25 (ya es un número, se puede usar en cálculos)
```

### Conversión String → double con Double.parseDouble()

Permite convertir texto que representa un número decimal en un valor de tipo double. Se usa para leer precios, salarios, horas u otros valores con decimales.

Ejemplo:

```
String valor = "15.75";
double precio = Double.parseDouble(valor);
// precio = 15.75 (número decimal listo para calcular)
```

### Conversión Explícita (Casting)

El casting fuerza la conversión entre tipos compatibles cuando Java no la realiza automáticamente. Se escribe el tipo destino entre paréntesis antes del valor.

Ejemplo:

```
double promedio = 9.8;
int nota = (int) promedio;
// nota = 9 (se trunca la parte decimal)
```

### Tipos de conversión y riesgos

| Tipo                          | Descripción                                                                                          | Ejemplo                                                    |
|-------------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| <b>Implícita (automática)</b> | Java la realiza sola sin necesidad de indicarla. No hay pérdida de datos. De tipo menor a mayor.     | int x = 5; double d = x; // d = 5.0                        |
| <b>Explícita (casting)</b>    | El programador la indica. Puede haber pérdida de información al ir de tipo mayor a menor.            | double n = 15.99; int e = (int) n; // e = 15 (pierde 0.99) |
| <b>String → int/double</b>    | Mediante métodos estáticos de la clase Integer y Double. Lanza excepción si el texto no es numérico. | Integer.parseInt("25")<br>Double.parseDouble("9.8")        |

Riesgo de pérdida de información:

```
double numero = 15.99;
int entero = (int) numero;
// Resultado: 15 ← Se pierde el decimal 0.99
```

La parte decimal se descarta (trunca), no se redondea. Esto puede generar errores en cálculos financieros o estadísticos.

## Palabras Clave del Manejo de Excepciones

| Palabra        | ¿Para qué sirve?                                                                       | Ejemplo mental                                           |
|----------------|----------------------------------------------------------------------------------------|----------------------------------------------------------|
| <b>try</b>     | Encierra el código que puede fallar. Si ocurre un error, Java salta al bloque catch.   | Intentar hacer algo arriesgado.                          |
| <b>catch</b>   | Captura y controla el error. Recibe el objeto de excepción para manejarlo.             | Si falla, responder con un mensaje controlado.           |
| <b>finally</b> | Se ejecuta siempre, ocurra o no una excepción. Ideal para cerrar recursos.             | Cerrar o finalizar el proceso sin importar el resultado. |
| <b>throw</b>   | Lanza una excepción manualmente desde el código. Permite crear errores personalizados. | El programador provoca el error intencionalmente.        |
| <b>throws</b>  | Declara en la firma del método que puede lanzar cierta excepción. Avisa al llamador.   | Este método puede fallar; quien lo llame debe manejarlo. |

## Conclusiones

- Las excepciones en Java permiten escribir programas más robustos y seguros, separando el manejo de errores del flujo normal del código.
- Los bloques try-catch son indispensables en cualquier punto donde el usuario ingresa datos, ya que el ser humano comete errores al tipear.
- La diferencia fundamental entre un error lógico y una excepción radica en que el error lógico no detiene el programa pero produce resultados incorrectos, mientras que la excepción sí interrumpe la ejecución si no se maneja.
- Las conversiones `String → int` y `String → double` con `parseInt()` y `parseDouble()` son esenciales en aplicaciones de consola y siempre deben ir dentro de try-catch para protegerse de entradas no numéricas.
- El casting explícito (`int`) trunca decimales sin redondear, lo cual puede causar pérdida de información si no se tiene en cuenta en cálculos de precisión.
- Usar tipos específicos de excepciones (`NumberFormatException`, `NullPointerException`, etc.) en lugar del genérico `Exception` hace el código más legible y facilita el diagnóstico de errores.